

03 — Plotting

Quick decision guide — which plot do you need?

Situation	Plot type	Section
See if data is linear/quadratic	Scatter plot	1
Show regression fit + data	Scatter + line	2
Show prediction uncertainty	Uncertainty band	3
Classification: two classes	Colored scatter	4
Classification: show decision boundary	Scatter + line/contour	5
PCA: 2D projection	Scatter 2D	6
Show cost convergence	Line plot	7
Histogram vs PDF (Problem 2 style)	hist + plot	8

```
In [4]: import numpy as np
import matplotlib.pyplot as plt

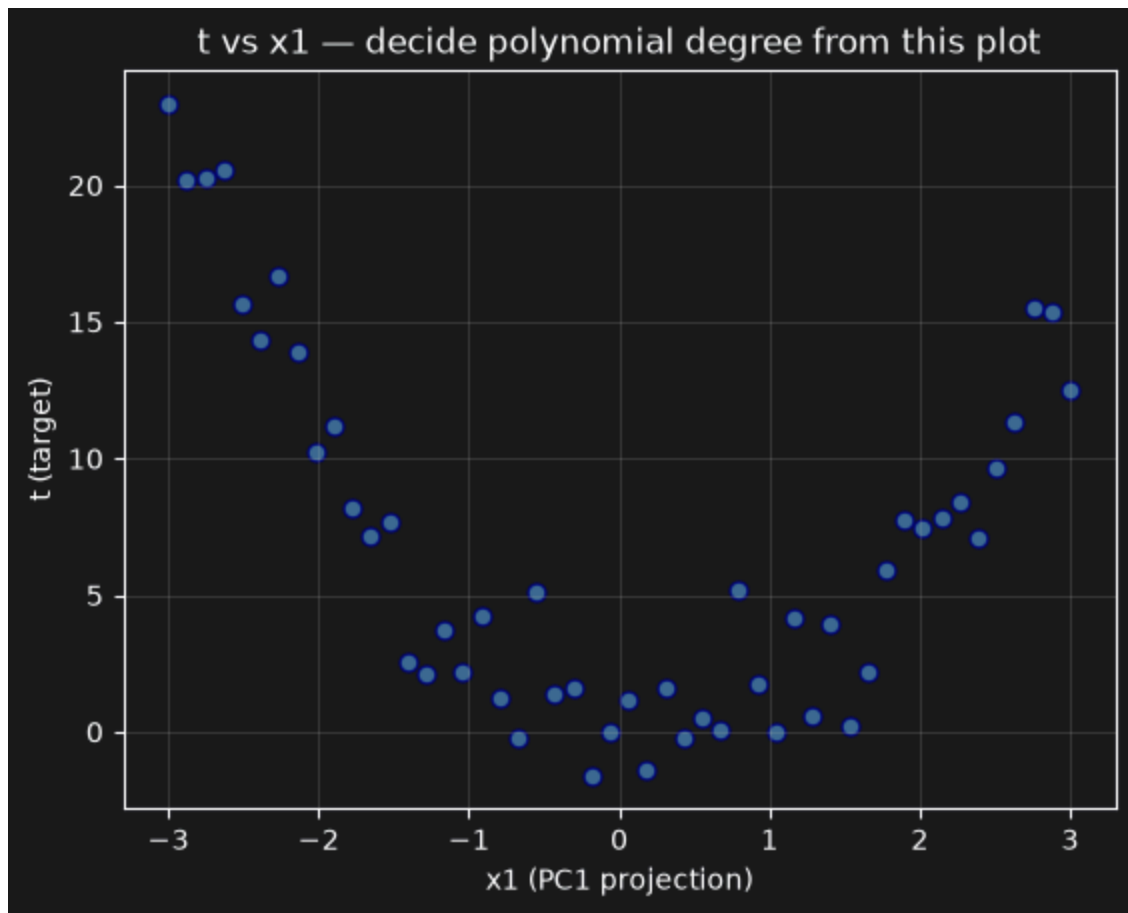
# --- dummy data to make all cells runnable ---
np.random.seed(42)
x = np.linspace(-3, 3, 50)
t = 2*x**2 - x + 1 + np.random.randn(50)*2
t_pred = 2*x**2 - x + 1
```

1. Scatter plot — look at the data

Use this first in any problem to understand the shape of the relationship.

```
In [5]: plt.scatter(x, t, color='steelblue', edgecolors='navy', s=40, alpha=0.8)
plt.xlabel('x1 (PC1 projection)') # Label your axes!
plt.ylabel('t (target)')
plt.title('t vs x1 — decide polynomial degree from this plot')
plt.grid(True, alpha=0.3)
plt.show()

# How to read it:
#   straight line → degree 1 (linear)
#   one bend (U/n) → degree 2 (quadratic) ← most common in exam
#   two bends (S) → degree 3 (cubic)
```

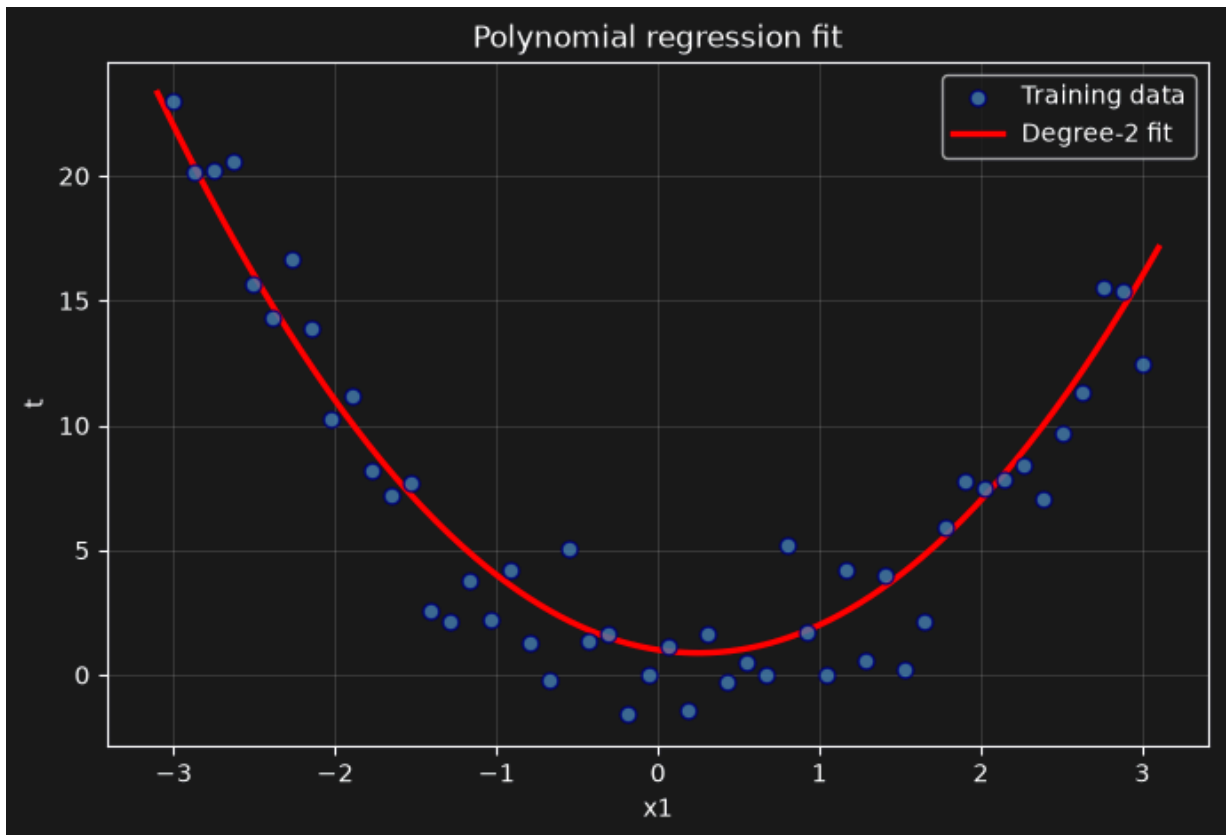


2. Regression fit — data + fitted curve

Show training data as dots, the regression line/curve as a smooth line.

```
In [7]: # Smooth line for the fit (300 points from min to max)
x_line = np.linspace(x.min() - 0.1, x.max() + 0.1, 300)
t_line = 2*x_line**2 - x_line + 1 # replace with: build_design_matrix(x_line, D

plt.figure(figsize=(8, 5))
plt.scatter(x, t, color='steelblue', edgecolors='navy', s=40, alpha=0.8,
            zorder=5, label='Training data')
plt.plot(x_line, t_line, 'r-', linewidth=2.5, label='Degree-2 fit')
plt.xlabel('x1')
plt.ylabel('t')
plt.title('Polynomial regression fit')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



3. Uncertainty band — fit + RMSE shading

The exam asks: "Determine the expected uncertainty of the predictions." Plot $\pm 1\sigma$ (RMSE) and $\pm 2\sigma$ bands around the fit.

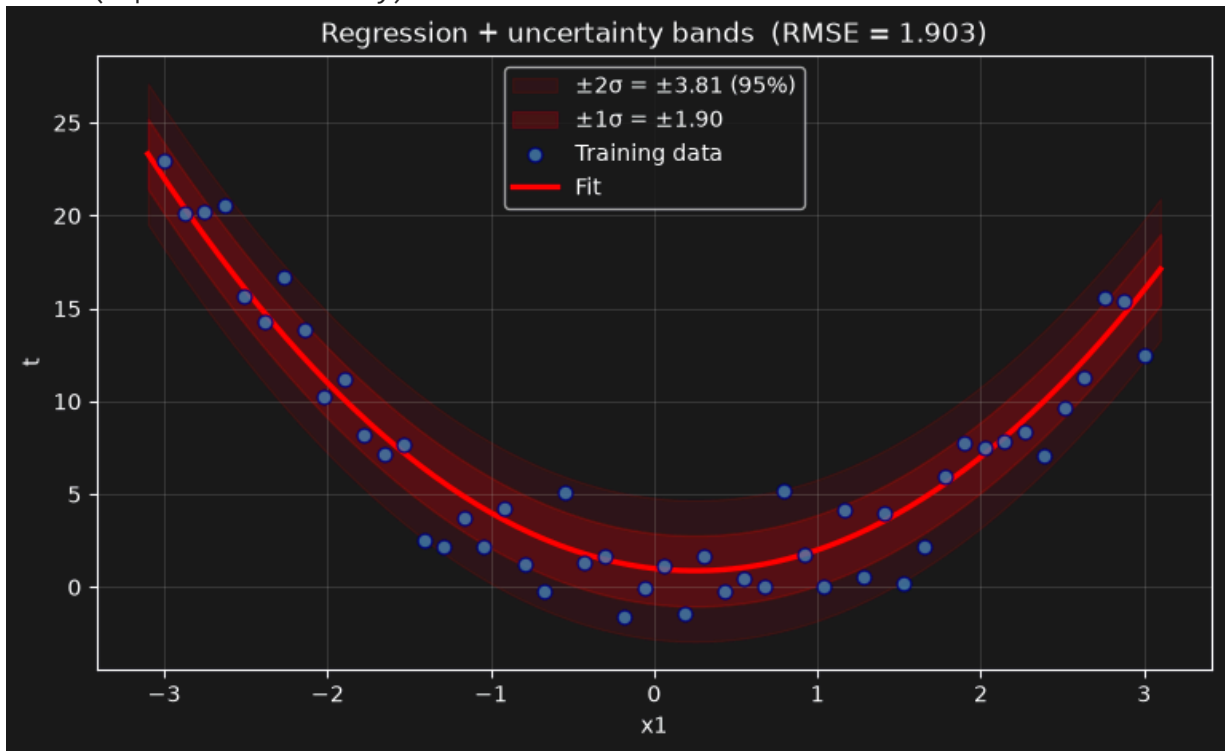
```
In [8]: # compute RMSE first
sigma_pred = np.sqrt(np.mean((t - t_pred)**2)) # RMSE on training data
print(f'RMSE (expected uncertainty): ±{sigma_pred:.3f}')

x_line = np.linspace(x.min() - 0.1, x.max() + 0.1, 300)
t_line = 2*x_line**2 - x_line + 1 # your fit

plt.figure(figsize=(9, 5))
# ±2σ band (lighter, wider)
plt.fill_between(x_line, t_line - 2*sigma_pred, t_line + 2*sigma_pred,
                 alpha=0.1, color='red', label=f'±2σ = ±{2*sigma_pred:.2f} (95%)')
# ±1σ band (darker, narrower)
plt.fill_between(x_line, t_line - sigma_pred, t_line + sigma_pred,
                 alpha=0.2, color='red', label=f'±1σ = ±{sigma_pred:.2f}')
# data and fit on top
plt.scatter(x, t, color='steelblue', edgecolors='navy', s=40, alpha=0.8,
            zorder=5, label='Training data')
plt.plot(x_line, t_line, 'r-', linewidth=2.5, zorder=4, label='Fit')
plt.xlabel('x1')
plt.ylabel('t')
plt.title(f'Regression + uncertainty bands (RMSE = {sigma_pred:.3f})')
```

```
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

RMSE (expected uncertainty): ± 1.903



4. Classification scatter — two classes with different colors/markers

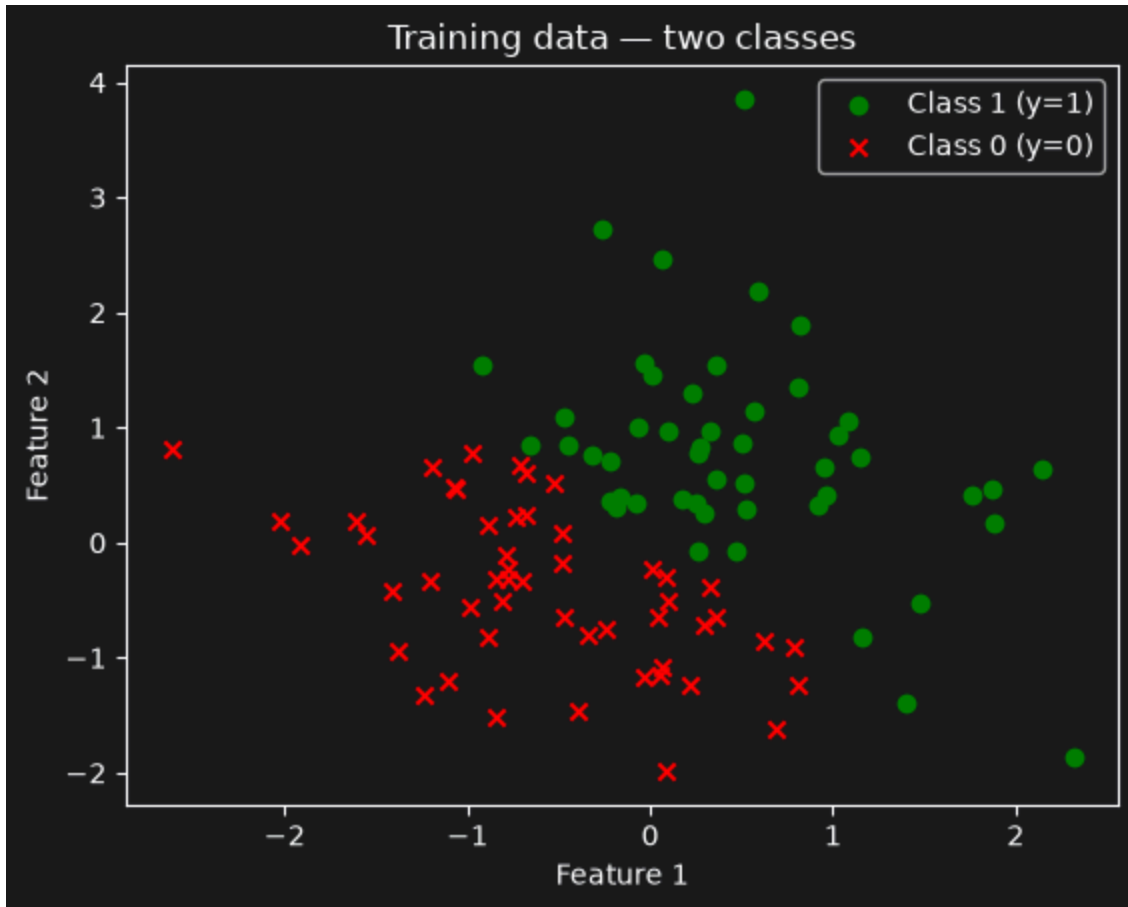
Used in logistic regression problems.

```
In [9]: # dummy classification data
X_cls = np.random.randn(100, 2)
Y_cls = (X_cls[:, 0] + X_cls[:, 1] > 0).astype(int).reshape(-1, 1)

# separate into two groups
pos, neg = [], []
for i in range(Y_cls.size):
    if Y_cls[i] == 1:
        pos.append(X_cls[i, :])
    else:
        neg.append(X_cls[i, :])
pos = np.array(pos)
neg = np.array(neg)

plt.scatter(pos[:, 0], pos[:, 1], c='g', marker='o', label='Class 1 (y=1)')
plt.scatter(neg[:, 0], neg[:, 1], c='r', marker='x', label='Class 0 (y=0)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Training data - two classes')
```

```
plt.legend()
plt.show()
```



5a. Decision boundary — linear case

For logistic regression with 2 normalized features. Decision boundary is where $w_0 + w_1 \cdot x1_norm + w_2 \cdot x2_norm = 0$ → rearrange for $x2$.

```
In [15]: # dummy weights and normalization stats
w       = np.array([[0.5], [2.0], [1.5]]) # [w0, w1, w2]
mean    = np.array([0.0, 0.0])
std     = np.array([1.0, 1.0])
X_original = X_cls

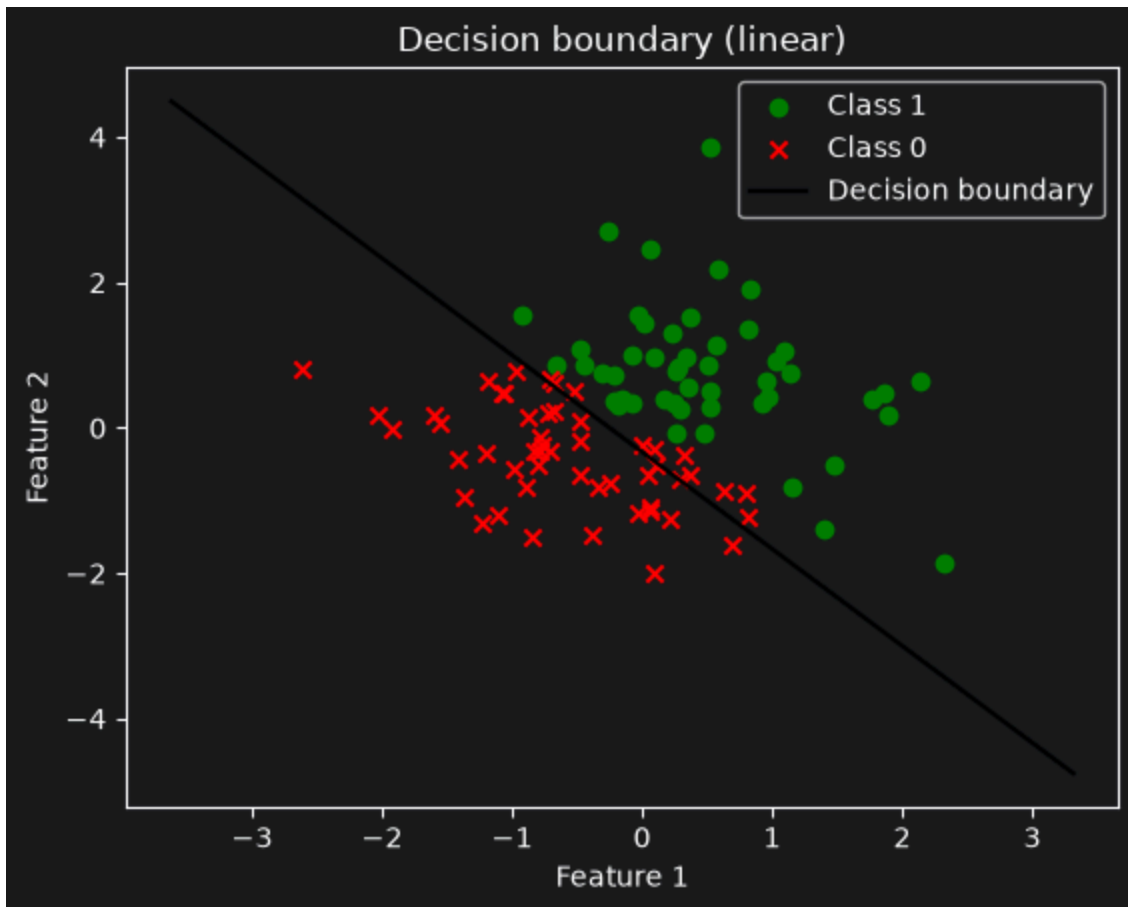
# x-range in original scale
x1_vals = np.array([X_original[:, 0].min() - 1, X_original[:, 0].max() + 1])

# normalize x1 range
x1_norm = (x1_vals - mean[0]) / std[0]

# solve for x2_norm: w0 + w1*x1_norm + w2*x2_norm = 0
x2_norm = (-w[0] - w[1]*x1_norm) / w[2]

# convert x2 back to original scale
x2_vals = x2_norm * std[1] + mean[1]
```

```
plt.scatter(pos[:, 0], pos[:, 1], c='g', marker='o', label='Class 1')
plt.scatter(neg[:, 0], neg[:, 1], c='r', marker='x', label='Class 0')
plt.plot(x1_vals, x2_vals, 'k-', label='Decision boundary')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Decision boundary (linear)')
plt.legend()
plt.show()
```



5b. Decision boundary — non-linear case (contour)

When features are polynomial-expanded (Lab04 style). Plot the contour where the model output = 0 (decision boundary).

```
In [20]: def mapFeature(X1, X2, deg):
# Build polynomial feature map: [1, x1, x2, x1^2, x1*x2, x2^2, ...]
try:
    m = X1.shape[0]
except:
    m = 1
X1 = np.array([X1])
X2 = np.array([X2])
out = np.ones((m, 1))
for i in range(1, deg + 1):
```

```

        for j in range(i + 1):
            term = ((X1**(i-j)) * (X2**j)).reshape(m, 1)
            out = np.hstack((out, term))
        return out

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

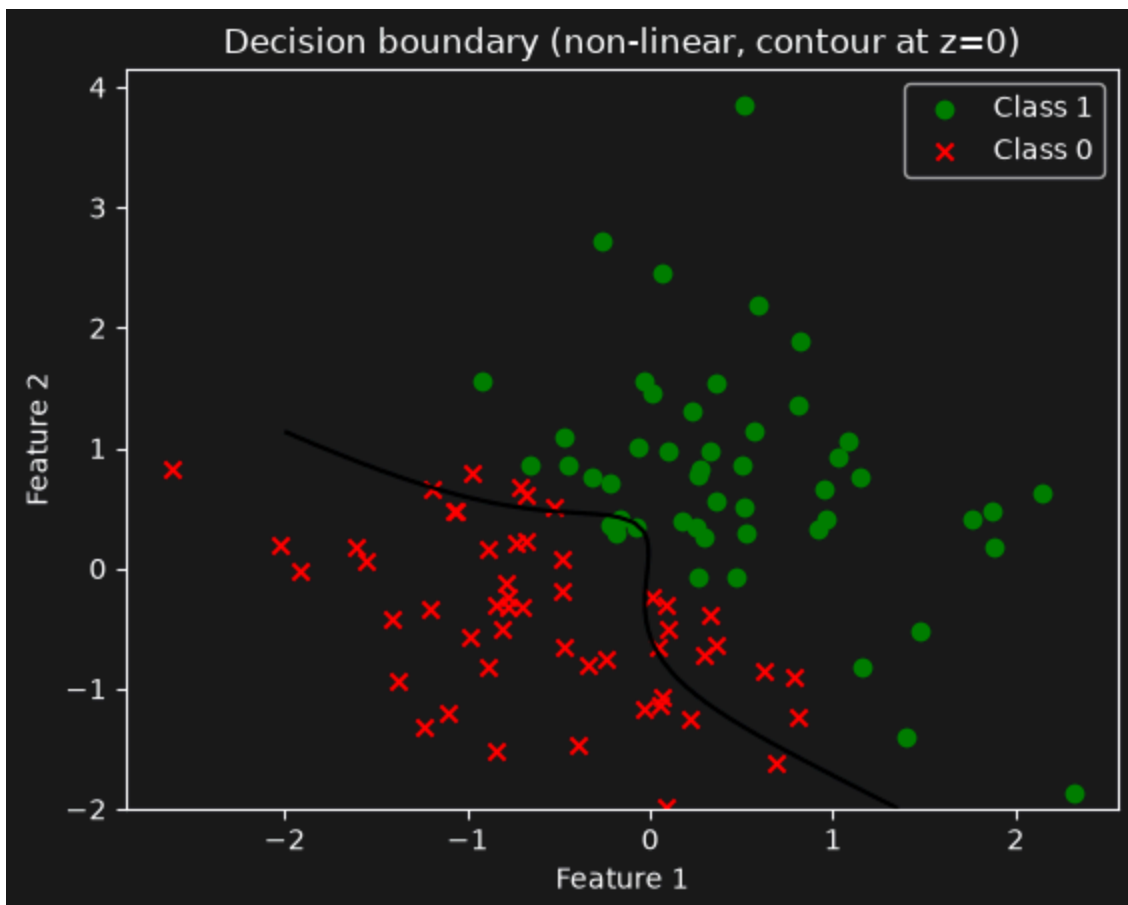
# dummy weights for degree=3 expanded features (10 features total)
w_nonlin = np.random.randn(10, 1) * 0.5

u_vals = np.linspace(-2, 2, 60)
v_vals = np.linspace(-2, 2, 60)
z = np.zeros((len(u_vals), len(v_vals)))

for i in range(len(u_vals)):
    for j in range(len(v_vals)):
        z[i, j] = (mapFeature(u_vals[i], v_vals[j], 3) @ w_nonlin)[0, 0]

plt.scatter(pos[:, 0], pos[:, 1], c='g', marker='o', label='Class 1')
plt.scatter(neg[:, 0], neg[:, 1], c='r', marker='x', label='Class 0')
plt.contour(u_vals, v_vals, z.T, levels=[0], colors='black') # contour at z=0
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Decision boundary (non-linear, contour at z=0)')
plt.legend()
plt.show()

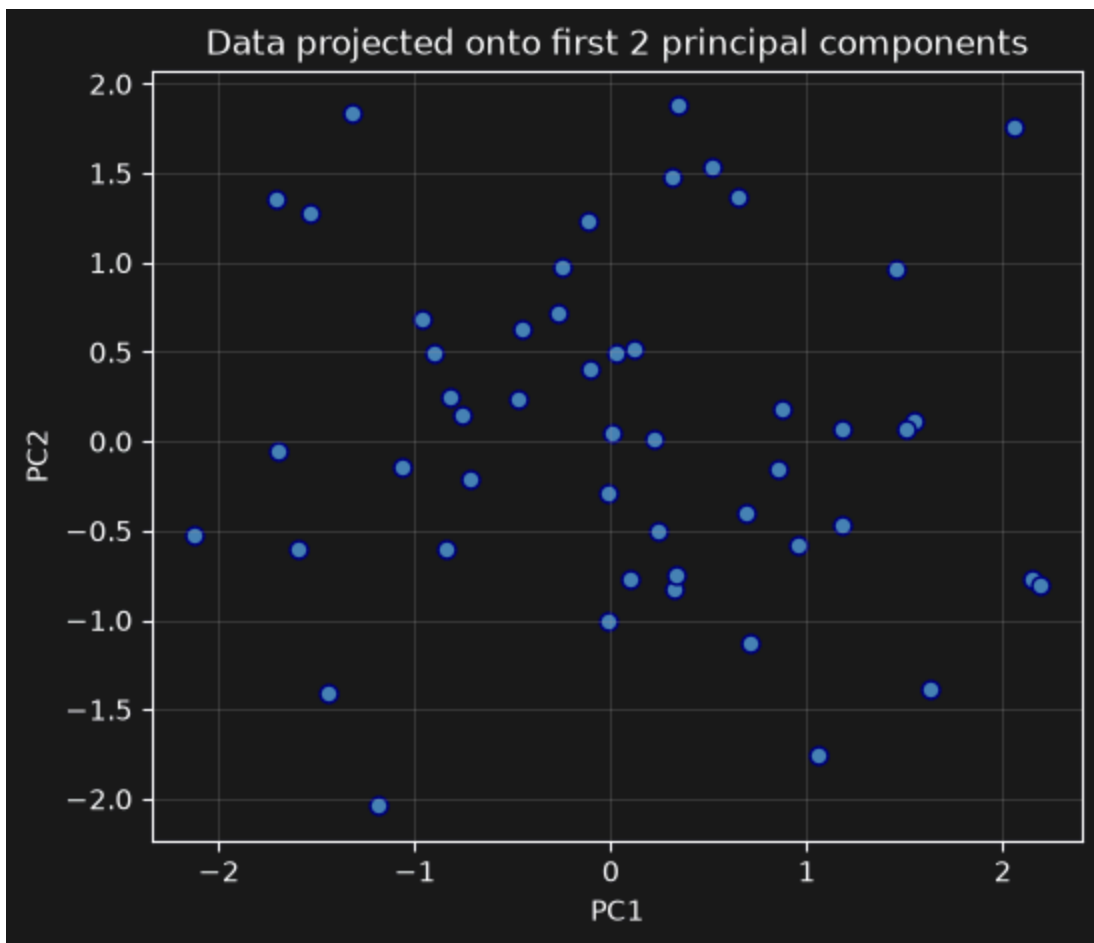
```



6. PCA 2D scatter — after projecting to 2 components

```
In [21]: # X_2d shape (m, 2) — two PCA components
# labels (optional) — if you have class labels
X_2d = np.random.randn(50, 2) # dummy

plt.figure(figsize=(6, 5))
plt.scatter(X_2d[:, 0], X_2d[:, 1], color='steelblue', edgecolors='navy', s=40)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('Data projected onto first 2 principal components')
plt.grid(True, alpha=0.3)
plt.show()
```



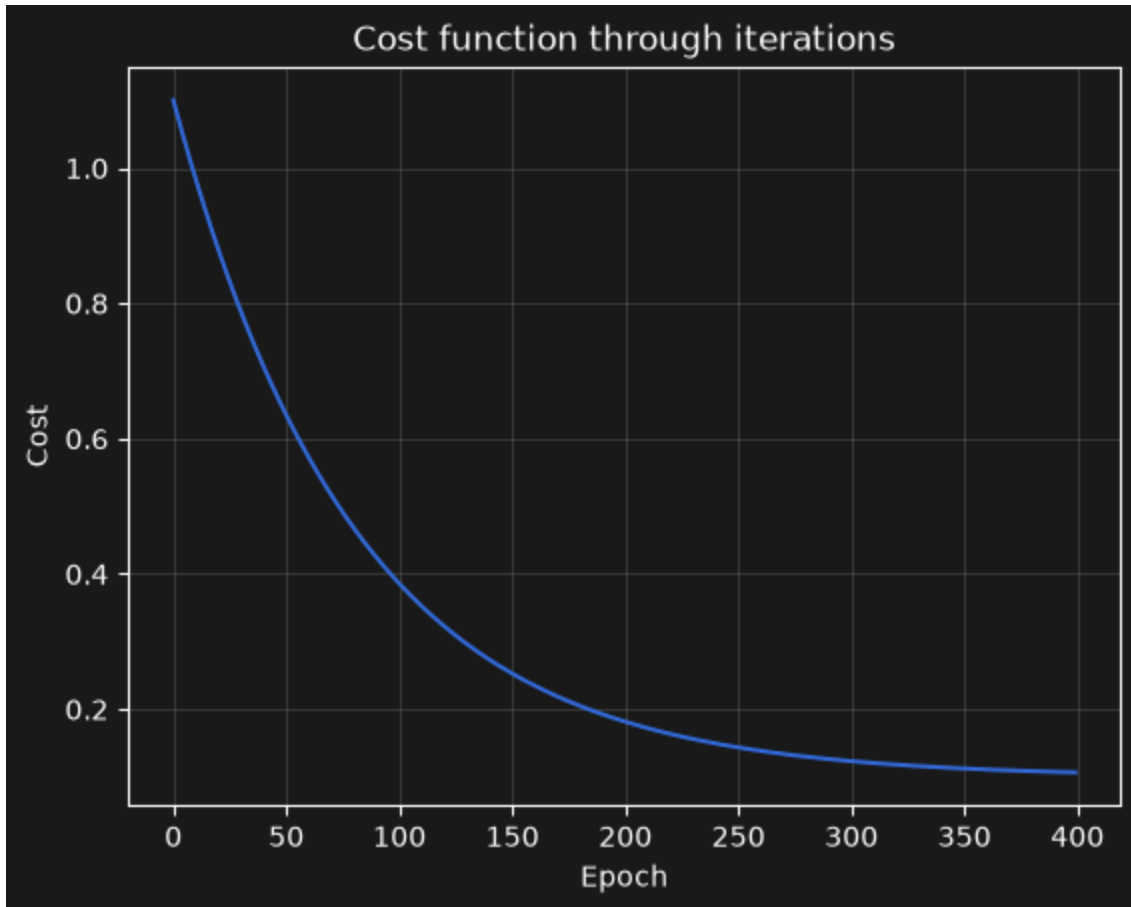
7. Cost convergence — show gradient descent is working

```
In [22]: # C_history is a list/array of cost values at each epoch
C_history = np.exp(-np.linspace(0, 5, 400)) + 0.1 # dummy
```



```
plt.plot(C_history)
plt.title('Cost function through iterations')
plt.xlabel('Epoch')
plt.ylabel('Cost')
plt.grid(True, alpha=0.3)
plt.show()
```

*# If it goes down and flattens → gradient descent converged ✓
 # If it diverges (goes up) → Learning rate too high*



8. Histogram vs theoretical PDF — Problem 2 style

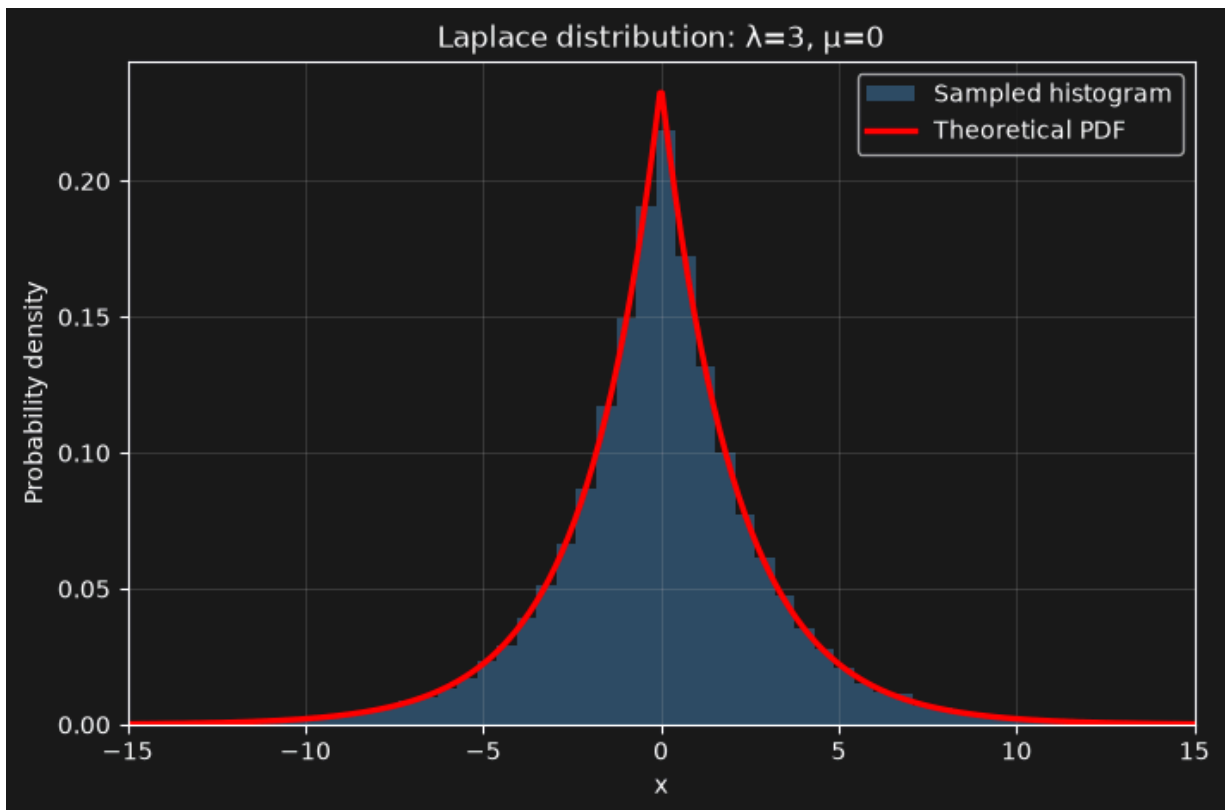
Show sampled distribution matches the theoretical one.

```
In [23]: # Example: Laplace distribution (from Problem 2)
lam = 3
mu = 0
N = 50000

rng = np.random.default_rng(seed=42)
u1 = rng.uniform(0, 1, N)
u2 = rng.uniform(0, 1, N)
s = np.where(u1 < 0.5, 1, -1)
x_samples = mu + s * lam * np.log(u2) / np.sqrt(2)
```

```
# theoretical PDF
x_plot = np.linspace(-15, 15, 500)
A      = 1 / (lam * np.sqrt(2))
pdf     = A * np.exp(-np.sqrt(2) * np.abs(x_plot - mu) / lam)

plt.figure(figsize=(8, 5))
plt.hist(x_samples, bins=80, density=True,          # density=True normalizes t
        alpha=0.5, color='steelblue', label='Sampled histogram')
plt.plot(x_plot, pdf, 'r-', linewidth=2.5, label='Theoretical PDF')
plt.xlabel('x')
plt.ylabel('Probability density')
plt.title(f'Laplace distribution:  $\lambda={\text{lam}}$ ,  $\mu={\text{mu}}$ ')
plt.legend()
plt.xlim(-15, 15)
plt.grid(True, alpha=0.3)
plt.show()
```



Matplotlib quick reference

```
plt.figure(figsize=(8, 5))          # set figure size
plt.scatter(x, y)                    # scatter plot
plt.plot(x, y, 'r-', linewidth=2)    # line plot
plt.fill_between(x, y1, y2, alpha=0.2) # shaded band
plt.contour(u, v, z, levels=[0])     # contour line (decision boundary)
plt.hist(x, bins=80, density=True)   # histogram normalized to area=1
plt.xlabel('label')
plt.ylabel('label')
plt.title('title')
```

```
plt.legend()
plt.grid(True, alpha=0.3)
plt.show() # always end with this
```

Common colors: 'steelblue', 'r' (red), 'g' (green), 'k' (black)

Common markers: 'o' (circle), 'x' (cross), 's' (square)

zorder : higher = drawn on top (use zorder=5 for data dots, zorder=4 for lines)